

KITE: KV-Invariant Transformer Expansion for Efficient Agentic LLM Scaling

Zhiheng Hu Yixun Wei Jian Zhou Yizhuang Zhou Ji Li Xing Chen
Yang Li Bojun Wang Yibo Zhu Xiangyu Zhang Daxin Jiang

StepFun 

Abstract

Scaling a language model is not only a question of final quality: the architectural choice determines how much computation is spent during training, prompt processing, and autoregressive decoding to achieve certain model quality. An ideal model architecture should lower all above computation costs to facilitate scaling to a larger model, while ensure the larger model indeed outperforms smaller baselines. We introduce *KV-Invariant Transformer Expansion (KITE)*, a scaling paradigm that achieves this goal. It trains the model from a smaller size to a larger size (*i.e.*, saving training costs via upcycling), while places newly added parameters in regions that do not affect attention KV. Consequently, during inference, prefilling KV only relies on the smaller part of the model, so the inference costs are saved. As a concrete instantiation, we present Step Scale Transformer (SST), a two-tower decoder in which one tower produces KV and the other reads them. At comparable cumulative training compute, SST, a 67B MoE model with 2.15B active body parameters per decode token, achieves lower training loss than 47B and 63B MoE Transformers with 1.48B and 2.02B active body parameters, respectively, while reducing estimated inference cost by 6.7% and 31.6%.

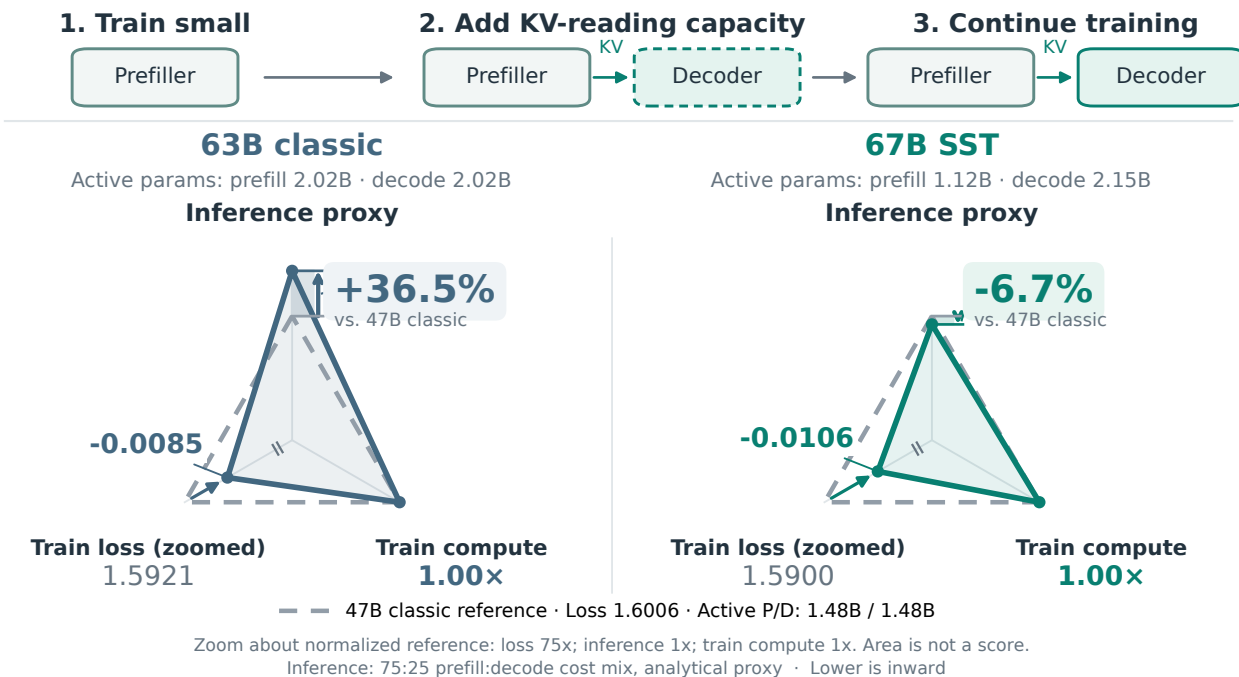


Figure 1: KITE: training route and observed trade-offs. Both panels use the same 47B transformer trained from scratch as the reference; lower is inward. Active counts exclude embedding/head. The inference proxy is $0.75p + 0.25d$, where p and d are bulk-prefill and decode active-body counts normalized to the reference.

1 Introduction

Large language models (LLMs) exhibit strong scaling behavior: increasing model scale continues to bring substantial gains in model capability [13, 14]. This property is both one of the most important blessings of LLMs and a curse, as larger models require enormous compute budgets for both training and inference.

Consequently, one of the central problems in LLM research is how to scale models at lower compute cost. This goal is not only about reducing expenditure; it enables a larger model to be trained or served under the same compute budget, yielding a stronger model. MoE feed-forward layers provide a successful example: replacing a dense FFN with sparse expert routing substantially reduces training and inference computation at a fixed parameter count, or, equivalently, allows model capacity to grow at a fixed compute budget [9, 20].

What is easy to miss in this success is that MoE reduces training and inference computation simultaneously. A saving on only one side may not improve the overall trade-off. For example, progressive upcycling trains a small model first and grows it into the target model partway through training, so much of the token budget runs on a cheaper graph [6, 28]. However, upcycled models are generally perceived to underperform same-size models trained from scratch, so the training saving is paid back at inference, where one serves the full target model yet obtains less than the full target quality. KV reuse pays on the other side. By letting deeper layers reuse the keys and values computed by shallower layers, YOCO family keeps prefill affordable as a model grows [8, 22, 25]; Yet removing KV-producing layers can sacrifice model quality; recovering the same quality may require a larger model or a longer training, which is effectively a training-cost loss. In both cases the gain on one axis is offset by a regression on the other. We therefore treat the two costs jointly: we pursue a scaling paradigm whose model quality, training cost, and inference cost all move in its favor.

This paper proposes *KV-Invariant Transformer Expansion (KITE)*, a model-scaling paradigm. Its core idea is that when the model is scaled, it is divided into two regions: the region that determines the key–value (KV) tensors, and the region that does not affect them. Scaling is carried out by expanding capacity in the region that does not affect the KV, which yields a good ratio of model intelligence to both training and inference cost. That is because the added capacity appears partway through training rather than from token zero, so the quality it buys comes at a favorable training cost. Meanwhile, this scaling does not affect the KV computation, so the prefill cost at inference remains unchanged.

KITE leverages the fact that prompt processing is significant in agentic workloads, where repeated model calls process accumulated context and new tool outputs [26, 27]. Even with prompt caching, substantial uncached input remains. Public OpenRouter traffic illustrates this input-heavy demand: across the six models in Table 1, uncached input exceeds output in token volume and accounts for most estimated uncached-input and output charges. Consequently, by saving on the prefill costs, KITE, like YOCO, produces models that are overall much cheaper to serve than classic transformers.

Table 1: Input demand in selected OpenRouter traffic. Input includes cached tokens; output includes reasoning tokens. Charge share is uncached-input charges divided by uncached-input plus output charges, excluding cache-read charges. Estimates aggregate available standard and Batch activity within August 22–September 20, 2026 (UTC), using September 21 base prices. Details appear in Appendix A.

Model	Input/output tokens	Uncached input/output tokens	Uncached-input charge share
GPT-6 Astra	106.43×	14.64×	74.6%
GPT-6 Astra Pro	63.15×	14.01×	73.8%
Claude Fable 5	63.38×	16.29×	76.7%
Claude Sonnet 5	59.88×	12.12×	70.8%
Gemini 3.7 Flash	37.79×	10.15×	67.4%
Gemini 3.8 Flash	46.14×	10.51×	67.9%

To instantiate KITE, we design a straightforward architecture: *Step Scale Transformer (SST)*. SST contains two same-depth towers that execute sequentially. The Prefiller produces layer-wise KV, which the Decoder reuses

to predict the next token. During generation, bulk prefill uses only the first tower, while the prompt-boundary prediction and decode use both. The source model is trained first; after expansion, both towers continue training jointly. Section 2 provides the architecture and execution details.

We spend the saved compute on scaling the model up moderately, just as MoE does. Compared with a Transformer baseline trained from scratch, this yields a net gain in the three-way trade-off among training cost, inference cost, and model intelligence. Figure 1 illustrates these trade-offs. KITE can also be used together with earlier successful designs such as MoE and hybrid attention; they are orthogonal and do not interfere with one another. Although SST is probably not the best KITE design, it is straightforward and suffices to demonstrate the positive effect of the paradigm. We are continuing to explore this paradigm, and we anticipate that more and more related designs can emerge.

To make the comparison clear, we build a scaling ladder across multiple model scaling paradigms. The tokenizer, data recipe, training protocol, and evaluation sets are kept identical, and we trace the cost-quality relationship of each scaling paradigm. A better scaling paradigm is one that reaches a lower training loss under a comparable training-FLOPs budget while also incurring a lower inference cost. The experiments show that the SST instantiation of KITE, expanded to 67B parameters, reaches an EMA-200 training loss of 1.5900, 0.0106 lower than 47B classic (1.6006) and 0.0021 lower than 63B classic (1.5921). At the final training endpoints, SST and 63B classic both use approximately 100% of the 47B baseline’s theoretical cumulative training FLOPs, including both SST training stages. With illustrative cost weights of 75% for prefill and 25% for decode, SST’s analytical inference-cost proxy is 6.7% lower than 47B classic and 31.6% lower than 63B classic (Figure 1). On downstream benchmarks, SST scores higher than both baselines on OpenBookQA, MMLU, GSM8K, MATH, HumanEval, MBPP, and BBH (Table 3).

2 KITE: KV-Invariant Transformer Expansion

We illustrate KITE through SST, a concrete instantiation that adds KV-reading capacity to a smaller model and continues training both parts jointly (Figure 2).

2.1 The design principle

Let a causal model have a Prefiller P and a Decoder D :

$$(M_t, z_t) = P(x_{1:t}; \theta_P), \quad \ell_t = D(x_t, z_t, M_t; \theta_D), \quad (1)$$

where M_t is reusable attention memory, z_t denotes optional Prefiller features at position t , and ℓ_t gives next-token logits. After an initial small-model training stage, expansion adds or enlarges D using the intermediate checkpoint, retaining the Prefiller graph and memory interface. Both parts then continue training.

The invariant is structural: neither weights nor KV values are frozen. The Decoder must not produce memory needed by future positions. To omit historical Decoder computation during generation, its retained positions must also be independent of omitted Decoder positions given the Prefiller outputs. These restrictions let prediction capacity grow without increasing *bulk prefill*, the prompt-wide KV construction performed by P . The final-position readout for the first output token and the Decoder’s work during decode remain necessary.

2.2 Step Scale Transformer

SST instantiates this principle with two Transformer towers [24]: the Prefiller (superscript P) and the Decoder (D), each with 18 layers in our experiments. The Prefiller executes first and produces layer-wise KV. Each Decoder block computes its own query and reads KV from the corresponding Prefiller layer:

$$q_{l,t}^D = Q_l^D(h_{l-1,t}^D), \quad a_{l,t}^D = \text{Attn}(q_{l,t}^D, K_l^P[I_l(t)], V_l^P[I_l(t)]) . \quad (2)$$

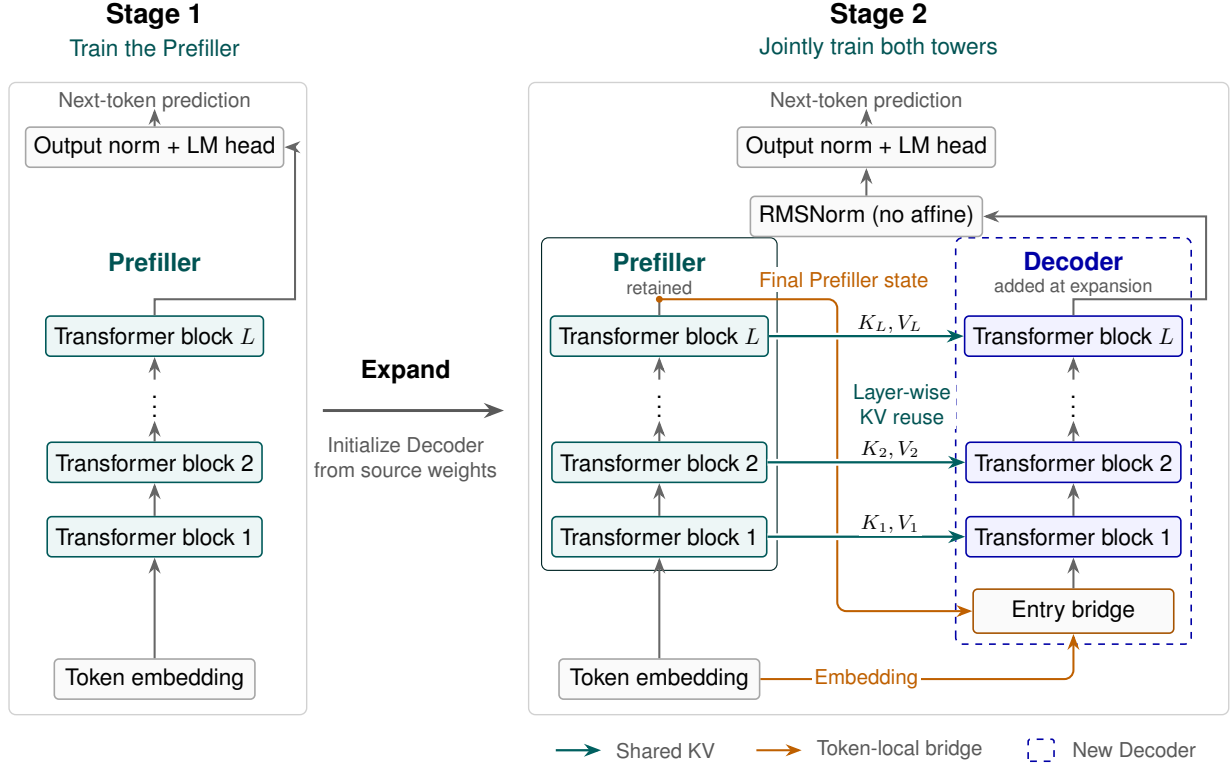


Figure 2: KITE instantiated with SST. Stage 1 trains the Prefiller; expansion adds a Decoder initialized from the source weights, and Stage 2 continues training both towers jointly. Within Stage 2, the Prefiller executes in full before the Decoder. Each Decoder block uses aligned Prefiller KV. A token-local entry bridge combines the separately normalized embedding and final Prefiller state. The final Decoder state is normalized and passed to the shared output norm and LM head for prediction.

Here, Q_l^D includes query projection and normalization, and $\mathcal{I}_l(t)$ specifies the positions allowed by the original causal full/sliding attention mask. Both towers retain the source model’s position IDs. The Decoder has independent block parameters; its residual and FFN/MoE operations are token-local.

In the expanded SST model shown on the right of Figure 2, a token-local entry bridge initializes the Decoder from the embedding and final Prefiller state. Prediction uses the Decoder’s final state. Gradients flow to the Prefiller through both the entry bridge and the reused KV. Given the Prefiller outputs, each Decoder position is independent of earlier Decoder states.

2.3 Training and inference

Training. After expansion, SST jointly trains both towers. The Prefiller processes the full sequence, and the Decoder computes every supervised position; gradients pass through the reused KV into the Prefiller. The initial small-model training stage precedes the addition of the Decoder.

Prefill. The Prefiller processes the complete prompt $x_{1:n}$ and produces the layer-wise KV cache. The Decoder needs to compute only the final prompt position n , attending to the Prefiller’s KV, to predict the first output token. Earlier Decoder positions can be omitted because its other operations are token-local.

Decode. Each new token passes through both towers. The Prefiller extends the KV cache, and the Decoder reads this KV to form the next-token prediction.

2.4 Decoder input and output

For the shared embedding/stem e_t and final Prefiller state $h_{L,t}^P$, SST uses the entry bridge

$$h_{0,t}^D = \text{RMS}_\epsilon(e_t) + \text{RMS}_\epsilon(h_{L,t}^P), \quad \text{RMS}_\epsilon(v) = \frac{v}{\sqrt{\text{mean}(v^2) + \epsilon}}. \quad (3)$$

The Decoder’s final state produces the next-token logits:

$$\ell_t = W_{\text{out}} \text{Norm}_{\text{out}}(h_{L,t}^D). \quad (4)$$

Here, Norm_{out} denotes the complete output normalization; its implementation is specified in Appendix D. The entry-bridge RMS operations have no trainable parameters and use $\epsilon = 10^{-5}$. The towers share one embedding, output normalization, and output head; embedding and head weights are untied.

3 Model construction and training

3.1 From the source to the expanded model

SST expands a 33.819B-parameter source into a 66.959B model while retaining the source tower as the Prefiller (Table 2). The evaluated route contains 167.98B source-stage tokens and 222.55B continuation tokens (Figure 3). Both stages contribute to the reported training budget.

The 47B and 63B classic-trained baselines are the next two scales above the 33.8B source in the same model family’s scaling ladder, obtained by increasing depth and width while retaining the MoE design and attention pattern. The 47B model provides an intermediate-scale reference, and the 63B model provides a reference close to SST’s 67B parameter count.

Table 2: Model configurations.

Setting	Source	SST	47B classic	63B classic
Layers	18	18 + 18	20	22
Hidden width	2304	2304	2560	2816
MoE layers	16	16 + 16	18	20
Routed / selected experts	512 / 8	512 / 8	512 / 8	512 / 8
Attention pattern	SSSF	SSSF	SSSF	SSSF
Total parameters (B)	33.819	66.959	46.727	62.691
Decode active body parameters (B/token)	1.120	2.155	1.477	2.016

Each tower starts with two dense FFN layers. SSSF is three sliding layers followed by one full-attention layer, with an SF remainder for 18/22 layers. Counts describe the effective architecture; Appendix B gives the geometry and counting convention.

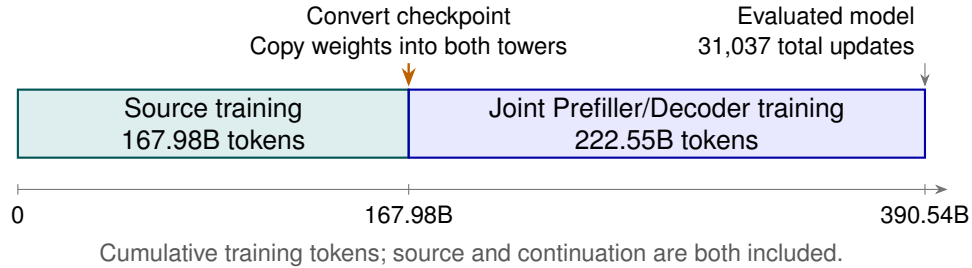


Figure 3: Training route to the evaluated SST checkpoint: 13,350 source steps followed by 17,687 continuation steps.

3.2 Expansion and continued training

We expand the source checkpoint into SST by copying each source layer into the corresponding layers of the Prefiller and the Decoder. The two towers share a single copy of the embedding, output normalization, and output head.

After expansion, both towers are trained jointly in BF16 using the source model’s Muon [17] and Adam [15] parameter groups. We retain the optimizer states for the Prefiller and the shared parameters and initialize fresh states for the Decoder. All groups continue on the source cosine learning-rate schedule, with the remaining decay extended over the continuation stage and no separate warmup for the Decoder. Appendix D gives the detailed settings.

3.3 Cumulative training compute

We compare the theoretical FLOPs of the effective architectures. For source and continuation token counts D_s, D_c and forward FLOPs/token f_s, f_c , we use

$$C_{\text{train}} = 3(D_s f_s + D_c f_c), \quad (5)$$

where the factor three approximates forward-plus-backward computation. The full cost of both training stages is included. The evaluated 47B classic checkpoint defines $B_{\text{ref}} \approx 5.16836 \times 10^{21}$ FLOPs. The final SST and 63B endpoints both use approximately 100% of this budget. Appendix C gives the theoretical accounting and training budgets.

4 Results

4.1 Training loss

SST’s loss rises at conversion and then decreases through continuation (Figure 4). At the final checkpoints, its training EMA-200 is 1.5900 after 390.54B tokens, versus 1.6006 for 47B classic after 443.16B tokens and 1.5921 for 63B classic after 334.62B tokens. SST therefore reaches lower training loss than both references at comparable cumulative training compute. Training budgets and loss reporting are given in Appendix C.

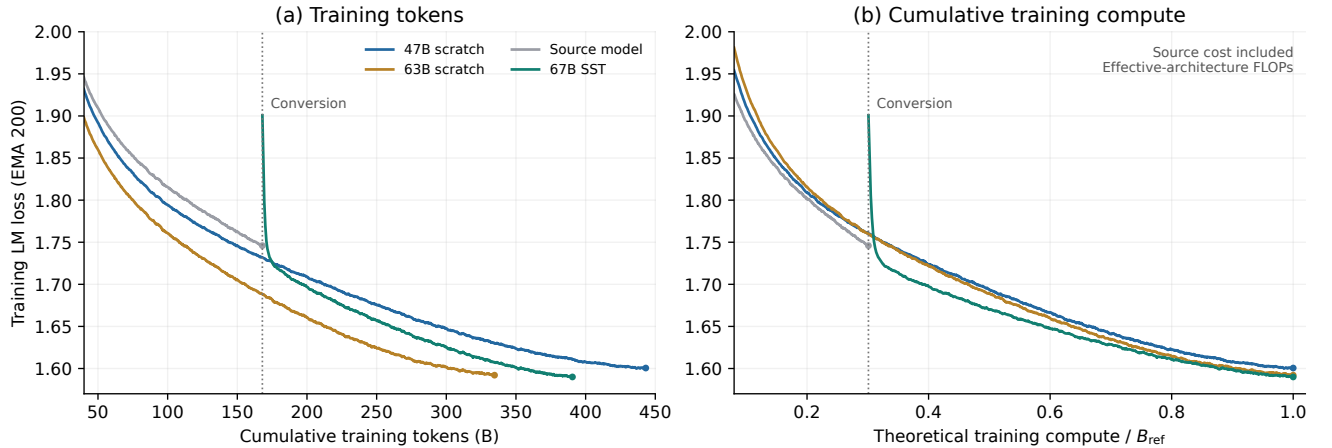


Figure 4: Training LM loss (EMA-200) against tokens and theoretical cumulative compute. Conversion occurs at 167.98B tokens ($0.301B_{\text{ref}}$).

4.2 Task-level evaluation

Table 3 reports scores on seven downstream tasks (OpenBookQA [18], MMLU [11], GSM8K [7], MATH [12], HumanEval [5], MBPP [2], and BBH [23]) at the final checkpoints of all three models. These are the same

checkpoints used for the training-loss and cumulative-compute comparisons. We also evaluate held-out token NLL on ARXIV using the same 1,024 frozen packed-4K samples for all three models.

Trained with an equal total of FLOPs, SST performs the best across the board. It shows that SST’s lowest training loss indeed translates to the best downstream performance. SST also has the lowest estimated inference cost in the input-heavy regime analyzed in Section 4.3.

Table 3: Scores (%) on seven downstream tasks and held-out ARXIV token NLL (lower is better), evaluated at comparable cumulative theoretical training FLOPs, including both training stages for SST. OpenBookQA combines the validation and test splits; MMLU uses the test split, and MATH uses the full test set.

Task	47B classic	63B classic	67B SST
OpenBookQA	68.50	71.00	75.00
MMLU	58.44	59.39	60.54
GSM8K	56.56	55.04	58.38
MATH	33.06	33.14	34.36
HumanEval (few-shot)	35.98	32.93	37.80
MBPP (3-shot)	50.40	50.80	51.40
BBH	50.36	52.56	52.74
ARXIV NLL ($\downarrow$)	1.4521	1.4424	1.4421

4.3 Inference cost across prefill–decode mixes

Figure 5 compares estimated inference cost across prefill:decode (P:D) cost mixes, using the proxy in Appendix A and active-body counts in Appendix B. All models use the same P:D weights, with costs normalized to 47B classic. SST breaks even with 63B at 13.4:86.6 and with 47B at 65.5:34.5, becoming cheaper as the prefill share increases. At 75:25, its estimated cost is 6.7% lower than 47B and 31.6% lower than 63B.

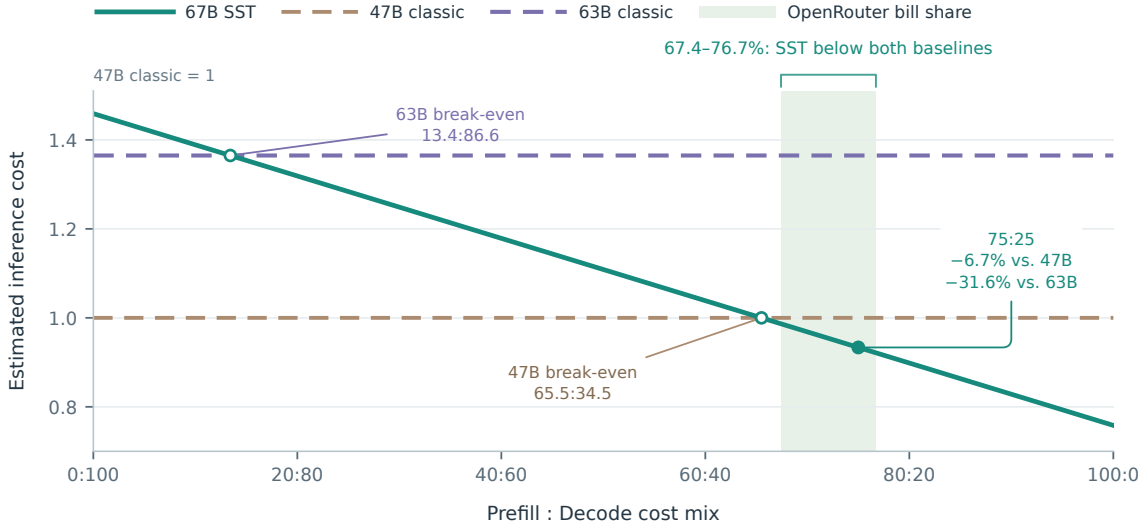


Figure 5: Estimated inference cost across P:D cost mixes, normalized to 47B classic. Shading spans the six-model OpenRouter reference range; open circles mark break-even points and the filled circle marks 75:25.

The six OpenRouter models in Table 1 have uncached-input bill shares of 67.4–76.7% (Appendix A). Using these shares as reference prefill weights, SST is cheaper than both baselines throughout the highlighted range: 1.3–7.9% below 47B and 27.7–32.5% below 63B. These are analytical cost estimates, not measured serving speedups.

5 Discussion

SST provides one concrete realization of KITE, rather than a unique architectural prescription. Its aligned layer-wise KV reuse, entry bridge, and Decoder-only readout are specific design choices. The central requirement is that the added capacity does not extend the KV-producing computation or introduce dependencies that require historical Decoder states. Figure 6 illustrates several connectivity choices within this framework.

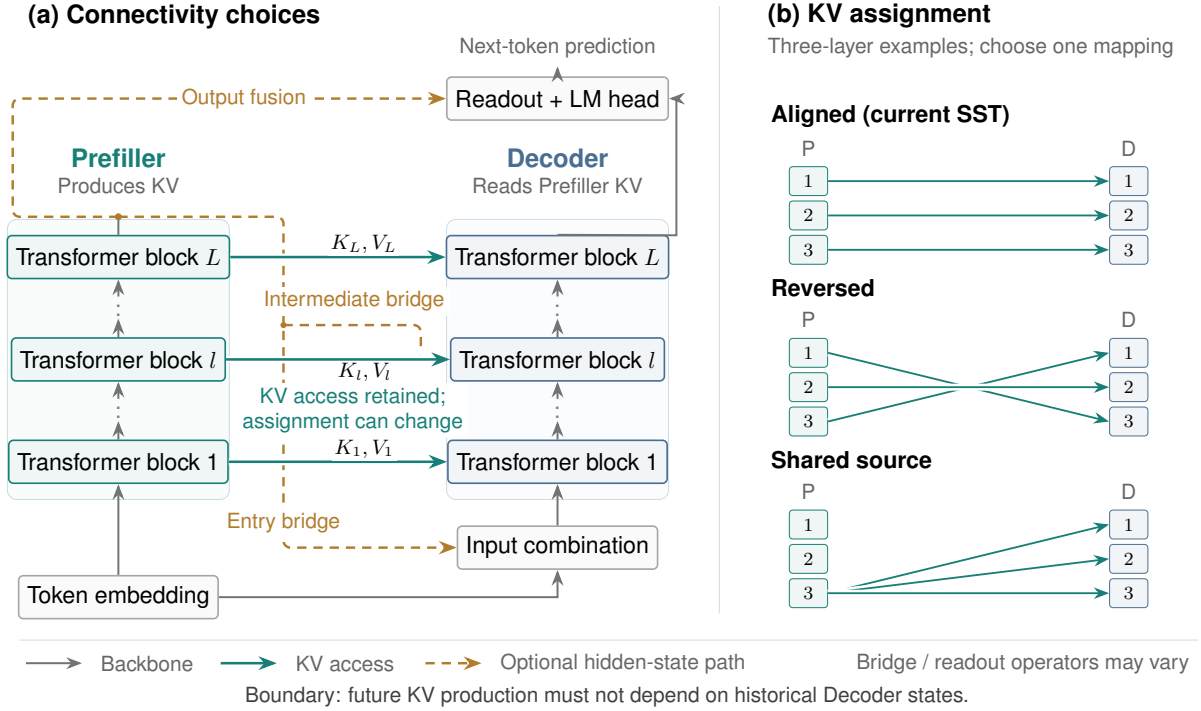


Figure 6: Connectivity choices within KITE. (a) An SST-style backbone with optional hidden-state connections at the Decoder entry, intermediate layers, and output readout. Dashed paths indicate design choices, not connections that must be enabled together. (b) Three illustrative KV assignments: aligned layer-wise reuse, reversed pairing, and reuse of one Prefiller KV source across multiple Decoder layers. The KV-producing computation remains on the Prefiller side; the illustrated choices are not assumed to have equal cost or quality.

The assignment between Prefiller KV and Decoder layers need not be one-to-one or order-preserving. For example, Decoder layers could read Prefiller KV in reversed layer order, or multiple Decoder layers could reuse a single KV source. Hidden-state connections offer another degree of freedom: Prefiller representations may enter the Decoder at its input or at intermediate layers, and may also contribute to the final readout. The bridge operators, as well as the depth and width of the two components, can likewise vary. These choices must preserve causal access and compatible interfaces, but need not have the same computational cost or model quality.

Our experiments explore only a limited part of this design space. The results reported in this paper establish the performance of the selected SST configuration, rather than identify an optimal connectivity pattern. Further exploration can examine how KV assignment, hidden-state connections, and capacity allocation affect the balance between model quality, training computation, and inference cost. KITE provides a common setting for studying these choices without fixing the architecture to the particular connections used in SST.

6 Related work

Net2Net studies function-preserving model growth [6]; SPARKLING examines signal preservation and symmetry breaking in width-progressive learning [28]. Dense-to-MoE upcycling reuses dense checkpoints to initialize sparse expert models [10, 16]. SST instead adds the Decoder to an already sparse source partway through training. Its aligned initialization changes the prediction graph and is followed by joint adaptation.

Multi-query attention [19] and grouped-query attention [1] reduce KV storage by sharing KV heads across query heads. Cross-layer attention extends sharing across adjacent layers [4]. These approaches reduce the stored attention state; KITE targets capacity expansion without extending the KV-producing path.

YOCO separates cache-producing and cache-consuming computation and exploits this separation for prefill early exit [22]. SST applies this execution pattern to staged model growth. YOCO’s cross-decoder shares a global KV representation; SST reads aligned layer-wise KV and is constructed through source-to-target expansion.

Mixture-of-Recursions explores recursive KV sharing, reusing the first recursion’s KV across subsequent recursions [3]. From an architectural perspective, such fixed-KV looping can be viewed as a parameter-tied special case of the family underlying KITE. SST instead uses independent Prefiller and Decoder parameters, expanding capacity during training without extending the KV-producing path.

7 Conclusion

KITE expands a model during training by adding capacity that reuses KV, leaving bulk-prefill computation source-sized. SST provides a simple instance of this route. With both training stages counted, it reaches lower training loss than the 47B and 63B classic baselines at comparable theoretical training compute. It also scores higher on several downstream tasks. Relative to the 47B baseline, SST trades lower bulk-prefill computation for higher decode computation; its inference-cost proxy is lower in prefill-heavy workloads. The larger 63B reference has a higher inference proxy and slightly higher training loss. These observations support further investigation of the combined scaling route, without establishing a scaling law, a causal benefit from each ingredient, or measured serving speedup.

A OpenRouter workload statistics and inference proxy

Data and coverage. Table 1 summarizes six selected models using public OpenRouter daily activity retrieved on September 21, 2026. We aggregate available standard and Batch activity within August 22–September 20 UTC, without extrapolating missing days. Both Astra variants have 17 observed days (September 4–20), and Gemini 3.8 Flash has 19 (September 2–20). The other models have 30 standard-activity days; Claude Fable 5 has only 10 Batch days (August 22–31). This selected traffic is not separately labeled as agentic usage.

Token and charge accounting. Let P be total input tokens, C cache-read tokens, $U = P - C$ uncached input tokens, and D output tokens, including reasoning. The two token ratios are P/D and U/D . For each variant v , let a_v and o_v be the September 21 base input and output prices per million tokens. We estimate

$$A = 10^{-6} \sum_v U_v a_v, \quad O = 10^{-6} \sum_v D_v o_v, \quad s_{\text{uncached}} = \frac{A}{A + O}. \quad (6)$$

Standard and Batch charges are calculated separately before summation. Cache-read charges are excluded from both numerator and denominator. These are base-price estimates, not invoices; cache-write premiums, context-length and provider-specific pricing, historical price changes, and non-token fees are not modeled.

Sources. Activity comes from OpenRouter’s public model-activity API: [Astra](#), [Astra Pro](#), [Fable 5](#), [Sonnet 5](#), [Gemini 3.7 Flash](#), and [Gemini 3.8 Flash](#); Batch uses the same model identifier with `variant=batch`. Prices use the [OpenRouter catalog](#) snapshot; token definitions follow its [usage-accounting](#) and [prompt-caching](#) documentation. All snapshots were retrieved on September 21, 2026.

Relation to the inference proxy. These statistics motivate input-heavy inference but do not measure GPU costs. We apply the same prefill:decode cost weights to all models. For prefill weight w , the proxy is $wp + (1 - w)d$, where p and d are bulk-prefill and decode active-body parameter counts normalized to 47B classic. Thus $w = 0.75$ means a 75:25 P:D cost mix. Figure 1 uses this mix, giving 0.933 for SST and 1.365 for 63B classic, compared with 1 for 47B classic: SST is 6.7% below 47B and 31.6% below 63B. Figure 5 varies the same cost weights. These are analytical proxy comparisons, not measured serving speedups.

Table 4: Components underlying Table 1. Token counts are in billions; estimated charges are in millions of USD. U is uncached input, C cached input, and D output. Ratios use unrounded totals.

Model	U (B)	C (B)	D (B)	A (\$M)	O (\$M)
GPT-6 Astra	356.199	2232.435	24.323	3.5600	1.2135
GPT-6 Astra Pro	45.611	159.946	3.255	0.4554	0.1613
Claude Fable 5	159.115	460.103	9.770	1.5895	0.4822
Claude Sonnet 5	1126.045	4434.707	92.872	2.2514	0.9279
Gemini 3.7 Flash	2100.037	5722.111	207.001	1.5666	0.7570
Gemini 3.8 Flash	1299.288	4405.218	123.644	0.9714	0.4598

Standard input/output rates (USD per million tokens) are 10/50 for Astra, Astra Pro and Fable 5; 2/10 for Sonnet 5; and 0.75/3.75 for both Gemini models. Batch rates are half the corresponding standard rates.

B Parameter accounting

Parameter counts follow the configured tensor shapes, checked against the exported weight inventory. SST has two sets of Transformer blocks and one shared embedding, output norm, and output head. Only the Prefiller contains K/V projections and K normalization in the effective architecture. The entry and final-state RMS operations have no trainable parameters.

Table 5: Detailed geometry. Source/SST dimensions apply to each tower; both have two initial dense FFN layers. RoPE denotes rotary position embeddings [21].

Setting	Source / SST	47B classic	63B classic
Dense FFN width	4608	5120	5632
Routed / shared expert width	576 / 576	640 / 640	704 / 704
Head dimension	128	128	128
Full / sliding RoPE dimension	64 / 128	64 / 128	64 / 128
Full / sliding Q heads	24 / 40	24 / 40	32 / 48
KV heads	8	8	8

Total parameters count the effective architecture. Active body parameters exclude embedding/head and count Top-8 routed experts per MoE layer, shared experts, routers, and the remaining body weights needed for one token. These are per-token parameter counts, not measured execution costs.

Table 6: Architectural parameter counts in billions, rounded to three decimal places.

Quantity	SST	47B classic	63B classic
Total, including embedding/head	66.959	46.727	62.691
Embedding + output head	0.594	0.660	0.725
Total body	66.365	46.068	61.966
Decode active body	2.155	1.477	2.016
Prefiller-only active body	1.120	—	—

The exported vocabulary has 128,815 rows, without training padding. Excluding embedding and head from the active-body counts avoids treating the full embedding matrix as a per-token multiplication. The output head still contributes prompt-boundary and decode work and must be included in measured serving cost.

C Training budgets and loss reporting

SST’s total includes 167.98B source-stage tokens and 222.55B continuation tokens; both stages are included in its training compute.

Table 7: Training budgets at the final checkpoints. Theoretical training FLOPs are normalized to the 47B classic reference.

Model	Training tokens (B)	Relative training FLOPs
47B classic	443.16	1.000
63B classic	334.62	≈ 1.000
67B SST	390.54	≈ 1.000

The loss curves end at the evaluated final checkpoints. Training-loss curves use an exponential moving average with span 200 (`adjust=False`), computed from the recorded observations without interpolation. The EMA is restarted after SST conversion and computed before plot subsampling, with the final observation retained.

The evaluated 47B classic checkpoint defines $B_{\text{ref}} \approx 5.16836 \times 10^{21}$ FLOPs. Forward costs are approximately 3.887488B FLOPs/token for 47B classic, 5.148513B for 63B classic, 3.087252B for the source, and 5.410678B for SST. SST retains both towers’ attention and FFN/MoE computation but only one embedding, output head, and set of KV projections.

All forward coefficients use sequence length 4096, sliding-window size 512, and the padded training vocabulary of 128,896, with multiply-add counted as two FLOPs. They include attention projections and matrix products, embedding, output head, FFN/MoE, routers, and the estimator’s activation terms. Normalization, RoPE, residual and bridge operations, rematerialization, optimizer, loss/auxiliary loss, conversion, communication, and checkpoint I/O are excluded. The same theoretical accounting convention is used for all models.

D Training recipe and continuation scheduler

Continued training uses native BF16 parameters with Muon and Adam groups. Muon uses six Newton–Schulz steps, with packed attention split by head and GLU split by projection type; Adam uses $\epsilon = 10^{-15}$. The source peak base learning rate is approximately 1.03302×10^{-3} . The continuation scheduler uses a base weight decay of 0.15 with the inherited parameter-group multipliers. The source/SST/63B token batch is 12,582,912; the 47B token batch is 14,680,064.

SST continuation uses 128 NVIDIA H800 GPUs with expert parallelism of size 8, a sequence length of 4,096, and a global batch size of 3,072 sequences. It runs for 17,687 updates after 13,350 source-training updates, giving 31,037 updates in total.

The parameter-free RMS operations at the Decoder input and final state accumulate in FP32 and return the input dtype; the original learned output norm is applied after the final-state RMS operation.

Let u be the number of continuation tokens consumed, $U = 222,553,964,544$ the configured continuation horizon, and q_0 the source’s normalized cosine phase at conversion. SST uses

$$q(u) = q_0 + (1 - q_0) \min(u/U, 1), \quad (7)$$

$$\eta(u) = \eta_{\min} + (\eta_{\text{anchor}} - \eta_{\min}) \frac{1 + \cos(\pi q(u))}{1 + \cos(\pi q_0)}. \quad (8)$$

The source schedule warms up over 12.582912B tokens and decays until 335.9605248B tokens. Conversion occurs at 167.9818752B tokens. Thus q_0 is measured within the post-warmup decay interval, not by dividing the conversion position by the whole horizon. The learning-rate constants are

$$\begin{aligned} \eta_{\text{anchor}} &\approx 5.96548 \times 10^{-4}, \\ \eta_{\min} &\approx 1.03302 \times 10^{-4}. \end{aligned}$$

Parameter-group multipliers remain applicable to the common base learning rate.

References

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*, 2023. URL <https://arxiv.org/abs/2305.13245>.
- [2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732*, 2021. URL <https://arxiv.org/abs/2108.07732>.
- [3] Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyouon Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron C. Courville, and Se-Young Yun. Mixture-of-RecurSIONs: Learning dynamic recursive depths for adaptive token-level computation. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL <https://arxiv.org/abs/2507.10524>.
- [4] William Brandon, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan-Kelly. Reducing Transformer Key-Value Cache Size with Cross-Layer Attention. *arXiv preprint arXiv:2405.12981*, 2024. URL <https://arxiv.org/abs/2405.12981>.
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*, 2021. URL <https://arxiv.org/abs/2107.03374>.
- [6] Tianqi Chen, Ian Goodfellow, and Jonathon Shlens. Net2Net: Accelerating learning via knowledge transfer. In *International Conference on Learning Representations*, 2016. URL <https://arxiv.org/abs/1511.05641>.
- [7] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*, 2021. URL <https://arxiv.org/abs/2110.14168>.
- [8] DeepSeek. DeepSeek-V4.1-Flash: Pushing the limits of kv cache compression. DeepSeek-V4.1-Flash, <https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash>, 2026. Technical report and model card. Accessed September 22, 2026.
- [9] William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL <https://arxiv.org/abs/2101.03961>.
- [10] Ethan He, Abhinav Khattar, Ryan Prenger, Vijay Korthikanti, Zijie Yan, Tong Liu, Shiqing Fan, Ashwath Aithal, Mohammad Shoeybi, and Bryan Catanzaro. Upcycling large language models into mixture of experts. *arXiv preprint arXiv:2410.07524*, 2024. URL <https://arxiv.org/abs/2410.07524>.
- [11] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring Massive Multitask Language Understanding. *arXiv preprint arXiv:2009.03300*, 2020. URL <https://arxiv.org/abs/2009.03300>.
- [12] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring Mathematical Problem Solving With the MATH Dataset. *arXiv preprint arXiv:2103.03874*, 2021. URL <https://arxiv.org/abs/2103.03874>.
- [13] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022. URL <https://arxiv.org/abs/2203.15556>.
- [14] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. URL <https://arxiv.org/abs/2001.08361>.
- [15] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980*, 2014. URL <https://arxiv.org/abs/1412.6980>.
- [16] Aran Komatsuzaki, Joan Puigcerver, James Lee-Thorp, Carlos Riquelme Ruiz, Basil Mustafa, Joshua Ainslie, Yi Tay, Mostafa Dehghani, and Neil Houlsby. Sparse Upcycling: Training Mixture-of-Experts from Dense Checkpoints. *arXiv preprint arXiv:2212.05055*, 2022. URL <https://arxiv.org/abs/2212.05055>.
- [17] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, et al. Muon is Scalable for LLM Training. *arXiv preprint arXiv:2502.16982*, 2025. URL <https://arxiv.org/abs/2502.16982>.

- [18] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering. *arXiv preprint arXiv:1809.02789*, 2018. URL <https://arxiv.org/abs/1809.02789>.
- [19] Noam Shazeer. Fast Transformer Decoding: One Write-Head is All You Need. *arXiv preprint arXiv:1911.02150*, 2019. URL <https://arxiv.org/abs/1911.02150>.
- [20] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. *arXiv preprint arXiv:1701.06538*, 2017. URL <https://arxiv.org/abs/1701.06538>.
- [21] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. *arXiv preprint arXiv:2104.09864*, 2021. URL <https://arxiv.org/abs/2104.09864>.
- [22] Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. You only cache once: Decoder-decoder architectures for language models. *arXiv preprint arXiv:2405.05254*, 2024. URL <https://arxiv.org/abs/2405.05254>.
- [23] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, et al. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. *arXiv preprint arXiv:2210.09261*, 2022. URL <https://arxiv.org/abs/2210.09261>.
- [24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*, 2017. URL <https://arxiv.org/abs/1706.03762>.
- [25] WeChat AI. Building effective sparse MoE models with moderate resources. WeLM Blog, <https://welm.weixin.qq.com/en/posts/building-effective-sparse-moe-models-with-moderate-resources/>, 2026. Published January 21, 2026. Source of the KV-Mirror architecture.
- [26] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv preprint arXiv:2405.15793*, 2024. URL <https://arxiv.org/abs/2405.15793>.
- [27] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022. URL <https://arxiv.org/abs/2210.03629>.
- [28] Qifan Yu, Xinyu Ma, Zhijian Zhuo, Minrui Wang, Deyi Liu, Shiyi Zhan, Yiyuan Ma, Liang Xiang, Xingyan Bin, and Di He. SPARKLING: Balancing signal preservation and symmetry breaking for width-progressive learning. *arXiv preprint arXiv:2602.02472*, 2026. URL <https://arxiv.org/abs/2602.02472>.